

Performance Testing

Date	1 July 2026
Team ID	xxxxxx
Project Name	Rising Waters
Maximum Marks	5 Marks

Performance Testing

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Load Testing, Functional API Testing
Target Module / API	Flood Prediction API (/predict), User Login, Dashboard
Test Environment	Local System (Flask Application)
Test Date	01-07-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	User Login Authentication	50	120	All users should log in successfully with response time under 2 seconds.
2	Flood Prediction Request	100	180	AI model should generate predictions accurately without errors.
3	Weather API Data Retrieval	75	120	Weather data should be fetched successfully with minimal latency.
4	Multiple Prediction Requests	30	180	Application remains responsive with no failures

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.35 seconds	Pass	Average response remained within the target.
2	Response Time (Max)	< 5 seconds	3.82 seconds	Pass	Peak response observed during concurrent prediction requests.
3	Throughput (Req/sec)	> 45 req/sec	58 req/sec	Pass	System handled concurrent requests efficiently.
4	Error Rate	< 1%	0.25%	Pass	Minor timeout observed during API testing.
5	CPU Utilization	< 80%	67%	Pass	Stable CPU usage throughout testing.
6	Memory Utilization	< 80%	71%	Pass	Memory usage remained within acceptable limits.

Step 4: Observations & Analysis

Key Findings

- The application handled concurrent prediction requests successfully.
- Average response time remained below the target threshold.
- No critical errors or crashes occurred during testing.
- Prediction accuracy and application responsiveness remained consistent.

Bottlenecks Identified

- Slight increase in response time during simultaneous prediction requests.
- Initial model loading may cause a minor delay on the first request.

Optimization Steps Taken

- Reused the trained XGBoost model instead of retraining it for each request.
- Optimized database queries and backend routing.
- Improved code modularity and reduced redundant operations.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results:

1. Login or Sign-Up Page
2. Dashboard View
3. Prediction Lists

1.



Rising Waters

Flood Risk Prediction System

Email, Phone Number, or Username

Password

☐ Remember me

Forgot password?

Sign In Securely

or continue with

Sign in with Google

New here? Create an account →

2.

